

UTILE : NTQuerySystemInformation

Thursday, February 05, 2026 1:49 PM

Variables :

```
ULONG ReturnLen = 0;
PVOID SystemInfo = 0; //buffer qui va recevoir l'info
//on créer une variable qui va nous servir à libérer la mémoire alloué
PVOID pValueToFree = NULL;
PSYSTEM_PROCESS_INFORMATION SystemProcInfo = NULL; //processus cible
WCHAR* ProcName = L"svchost.exe";
DWORD PID = 0;
NTSTATUS STATUS;
```

//structure de la fonction :

```
// Function pointer
typedef NTSTATUS (NTAPI* fnNtQuerySystemInformation)(
SYSTEM_INFORMATION_CLASS SystemInformationClass,
PVOID SystemInformation,
ULONG SystemInformationLength,
PULONG ReturnLength
);
```

Ensuite on initialise un pointeur de fonction :

```
fnNtQuerySystemInformation pNtQuerySystemInformation = NULL;
```

Enfin on récupère l'adresse de cette fonction native :

```
pNtQuerySystemInformation =
(fnNtQuerySystemInformation)GetProcAddress(GetModuleHandle("ntdll.dll"),
"NtQuerySystemInformation");
```

Ensuite on doit récupérer la taille du buffer à allouer, pour ça on va lancer une première fois la fonction pour récupérer une erreur qui nous donnera la taille nécessaire :

```
STATUS = pNtQuerySystemInformation((SYSTEM_INFORMATION_CLASS)5, NULL, 0, &ReturnLen);
```

Maintenant qu'on connaît la taille on peut l'allouer avec HeapAlloc :

Ensuite on peut relancer la fonction NtQuerySystemInformation qui va effectuer une requête système pour obtenir des informations sur les processus en cours via NtQuerySystemInformation (en spécifiant ce qu'on souhaite récupérer avec SystemProcessInformation).

Suite à cette fonction, les informations sont stockées dans la variable SystemProcInfo.

Ensuite on va pouvoir énumérer chaque processus présent dans la machine avec un while(TRUE)

On va parcourir chaque processus qui sont des structures donc on va parcourir ces listes chaînées.

Dans la structure `_SYSTEM_PROCESS_INFORMATION`, on a un champ : **ImageName** de type `UNICODE_STRING` qui correspond au nom du process.

On a aussi le champ **NextEntryOffset** qui pointe sur la prochaine structure donc on va l'utiliser pour bouger sur une autre structure.

On va d'abord vérifier que le champ ImageName ne soit pas vide en le pointant sur l'élément Length de la structure de son type.

Ce qui nous donne :

```
If(SystemProcInfo->ImageName.Length != 0){
```

Et ensuite on compare la chaîne récupérée avec celle qu'on vise (ProcName)

Ce qui nous donne :

```
If(wcsncmp(SystemProcInfo->ImageName.Buffer, ProcName) == 0){
```

Ici si c'est égal à 0 alors la chaîne est similaire.

Alors on peut sauvegarder le PID de ce process dans notre variable PID qui retournera un HANDLE (comme on peut voir dans la structure)

Ce qui nous donne :

```
DWORD PID = (ULONG_PTR)(DWORD)SystemProcInfo->UniqueProcessId
```

On cast la **variable ProcInfo** car on veut récupérer une valeur que notre PID peut récupérer. Comme c'est un DWORD il ne peut pas récupérer un type HANDLE. (UniqueProcessId retourne un HANDLE)

Le type HANDLE est un pointeur vers une ressource système comme un processus ou un fichier. Sa taille dépend de l'architecture du système. Le PID est un entier de 32 bits donc on doit trouver un moyen de stocker le résultat dans cette variable qui retourne un type DWORD (32bits donc).

Pour ça, on va utiliser le type **ULONG_PTR** qui va permettre de traiter le **HANDLE** comme un **entier** de la taille d'un pointeur. Ensuite on utilise le cast **DWORD** pour stocker proprement dans le PID.

Bon ok, on a compris.

Maintenant que notre PID contient le bon process on peut breaker.

On peut tester si il ya un autre chemin vers la prochaine structure, si ce n'est pas le cas on break, sinon on va vers la prochaine structure en utilisant le champ NextEntryOffset

Quand on sort de la boucle on clean le HEAP avec HeapFree.

On peut checker si on a bien récupérer le PID avec un

```
if(PID == NULL){  
Return FALSE;  
else return PID;
```

Et enfin dans le main on peut appeler la fonction qui va juste nous récupérer le pid du process svchost.exe.

